

Manual Técnico

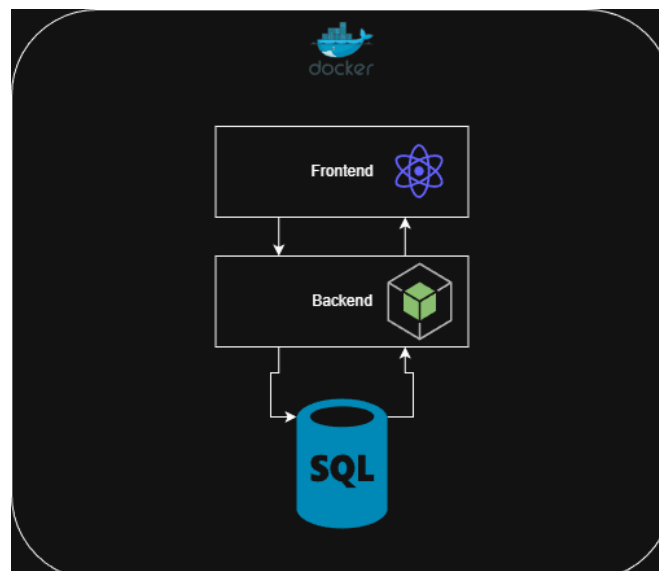
Diagrama Entidad Relacion

Para el esquema de base de datos, se manejan 3 tablas relacionadas de manera tal que puedan dar una lógica funcional a la aplicación en relación al consumo de datos.



Diagrama de Arquitectura

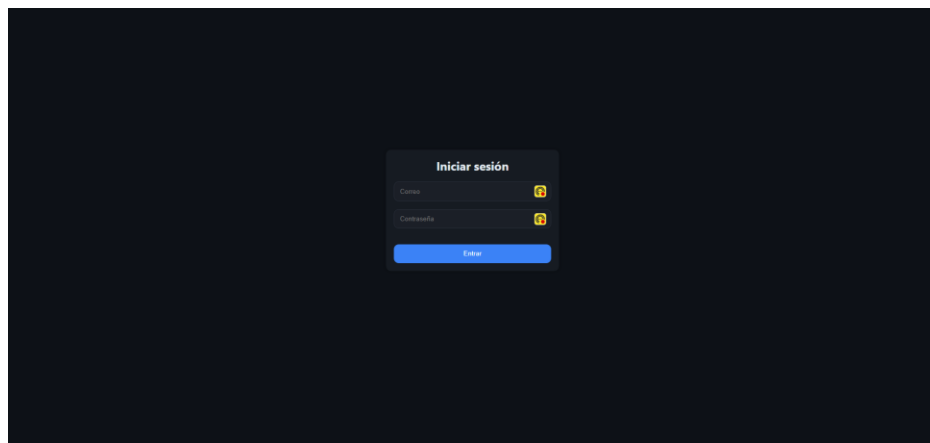
Se maneja una estructura básica tomando en consideración un backend nodejs, front ReactJS, un middleware para consumo de BD por Windows authentication por entorno local, y dockerizado para su globalización.



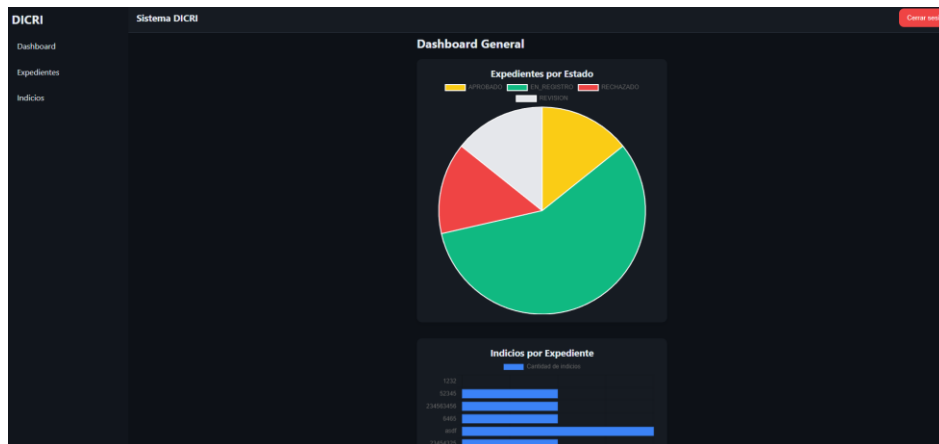
Resumen Técnico

La solución busca dar un sistema de ingreso a usuarios autorizados a nivel interno por BD (sin opción a registro de usuario voluntario por ser plataforma administrativa). También pretende manejar el ingreso de expedientes, ingreso de incidentes relacionados por expediente, gestión y actualización de estados por expediente y seguimiento por autorización o rechazo por usuarios coordinadores. Cuenta también con una sección básica para gráficos representativos en relación a los datos existentes y relevantes del aplicativo.

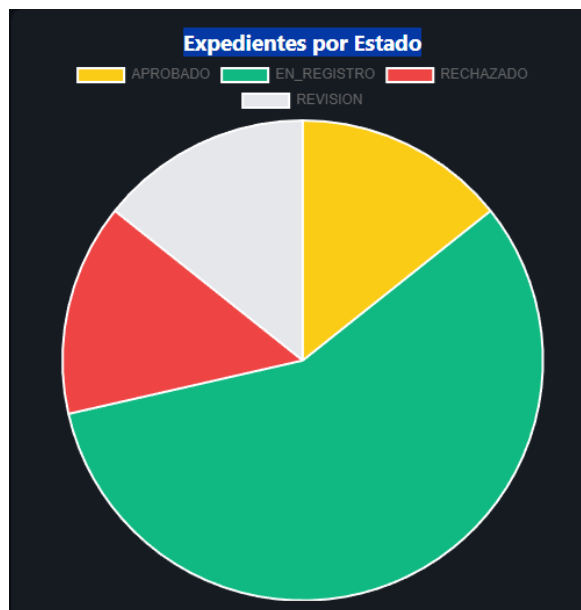
LoginPage



DashboardPage o Página de gráficos



Expedientes por estado: Esta muestra la totalidad porcentual de expedientes existentes y su magnitud de estados.



Indicios por expediente: Esta muestra la cantidad de indicios existentes por expediente en el sistema.



Expedientes por usuario: Esto muestra la cantidad de expedientes registrados por usuario en el sistema.

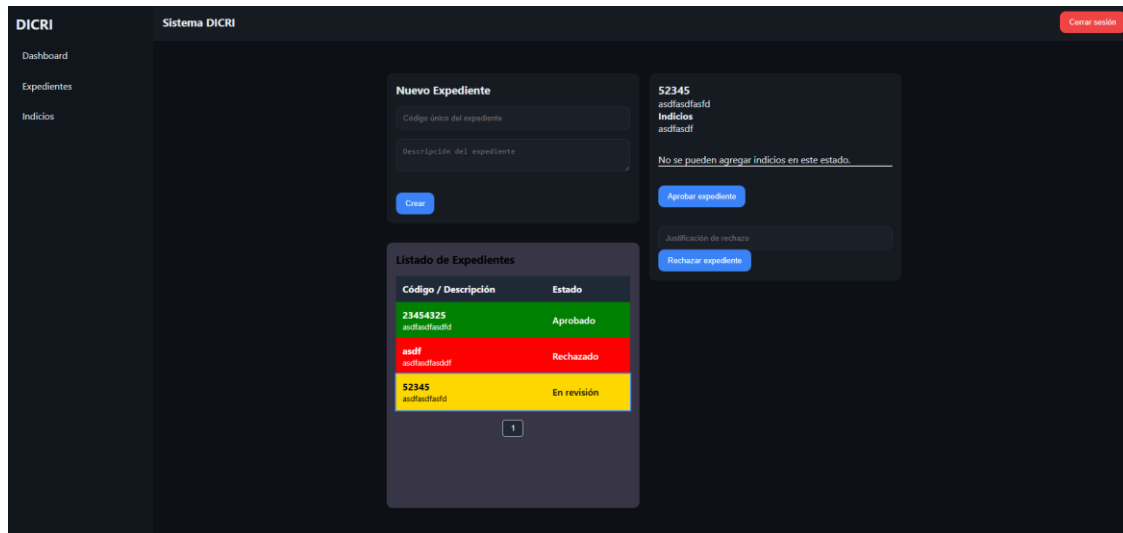


Expedientes Aprobados/Rechazados: Este muestra la cantidad de expedientes rechazados y aprobados en totalidad del sistema



ExpedientesPage (Coordinador)

Para la perspectiva de un usuario con rol Coordinador se puede agregar expedientes, ver la lista de expedientes en revisión, aprobados y rechazados, en el caso de los rechazados se muestra el motivo del rechazo, y en los estado en revisión se pueden seleccionar para su posterior autorización o rechazo.



Consumo de BD

Se establece por mssql configurando una instancia Docker con sql interno para conectar directamente con el contenedor de backend.

```
const sql = require('mssql');
const dotenv = require('dotenv');
dotenv.config();

const config = {
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  server: process.env.DB_HOST,
  database: process.env.DB_NAME,
  options: {
    encrypt: false, // para conexiones locales no necesitas encrypt
    trustServerCertificate: true
  }
};

let poolPromise = null;

async function getPool() {
  if (!poolPromise) {
    try {
      poolPromise = await sql.connect(config);
      console.log("Conexión exitosa a SQL Server desde otro contenedor");
    } catch (err) {
      console.error("Error conectando a SQL:", err);
      throw err;
    }
  }
  return poolPromise;
}

module.exports = { sql, getPool };
```

Ruta de repositorio

https://github.com/Mariocrv208/DICRI_MarioRodriguez.git

Implementacion de Token JWT (Middleware)

Se estructura la validación por JWT token para el acceso al sistema, configurando información sensible por configuraciones .ENV.

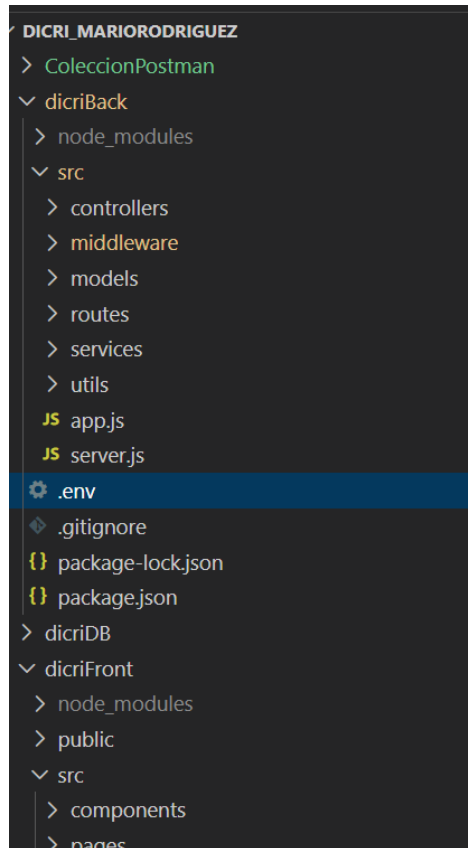
```
const jwt = require('jsonwebtoken');
const dotenv = require('dotenv');
dotenv.config();

exports.verifyToken = (req, res, next) => {
  const header = req.headers['authorization'];
  if (!header) return res.status(401).json({ message: 'No token provided' });
  const token = header.split(' ')[1];
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = payload;
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Token inválido' });    "inválido":
  }
};

exports.requireRole = (role) => (req, res, next) => {
  if (!req.user) return res.status(401).json({ message: 'No autorizado' });
  if (req.user.rol !== role && req.user.rol !== 'admin') {
    return res.status(403).json({ message: 'Acceso denegado' });    "Acceso":
  }
  next();
};
```

Esquema de proyecto (Estructura de carpetas)

Se estructura de manera tal que evite el llamado “Codigo espaguetti”, asegurando las buenas practicas de codificación.



Gitflow

Se maneja división en la estructura de ramas para asegurar el correcto flujo de git y correcto control de versiones.



Docker

Para el fin de la practica, se genero una colección para corresponder a backend, frontend y BD en instancia SQL.

☐	▼	●	dicri_mariorodriguez	-	-	-	2.39%	36 minutes ago	⚙️	■	⋮	🗑️
☐		●	dicri_sqlserver	352845dc23e4	mssql/server:2022-latest	1433:1433	0.5%	36 minutes ago	⚙️	■	⋮	🗑️
☐		●	dicri_frontend	6c101156d381	dicri_mariorodriguez-frontend	3000:5173	0.05%	36 minutes ago	⚙️	■	⋮	🗑️
☐		●	dicri_backend	386f0f57264f	dicri_mariorodriguez-backend	4000:4000	1.84%	36 minutes ago	⚙️	■	⋮	🗑️

Estas se interconectan entre si y están configuradas para complementarse en su funcionamiento.

Pruebas Unitarias

Se configuro el entorno Docker dicri_mariorodriguez-backend_tests-run-bf950362ccba con la finalidad de poder ejecutar las pruebas unitarias de algunos servicios backend dando salidas similares a:

```
Test Suites: 1 failed, 2 passed, 3 total
Tests:      1 failed, 4 passed, 5 total
Snapshots:  0 total
Time:       2.059 s
Ran all test suites.
```